

CVIA (8890) Assignment 1 – Image Segmentation Implementation

u3283999

Edge-Based Segmentation

For this implementation, Sobel edge detection and threshold was chosen due to simplicity and computational efficiency. It works by approximating the gradient of image intensity, allowing edge areas with strong intensity changes to be identified.

This method uses small convolution kernels, (in this case 3x3 matrices) to focus on local regions of the images. The kernels are designed to detect intensity changes in both the x and y axis, examples of these kernels can be found below.

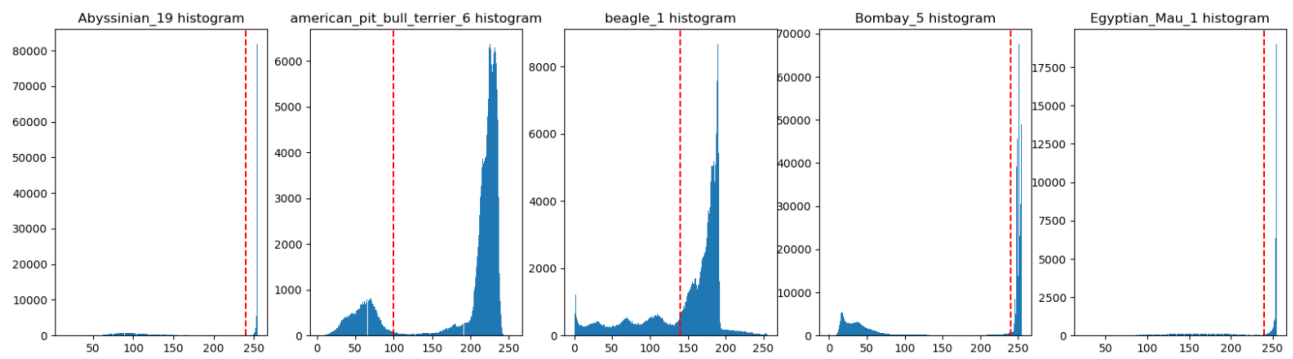
$$Gx = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix}$$

$$Gy = \begin{bmatrix} +1 & +2 & +1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$

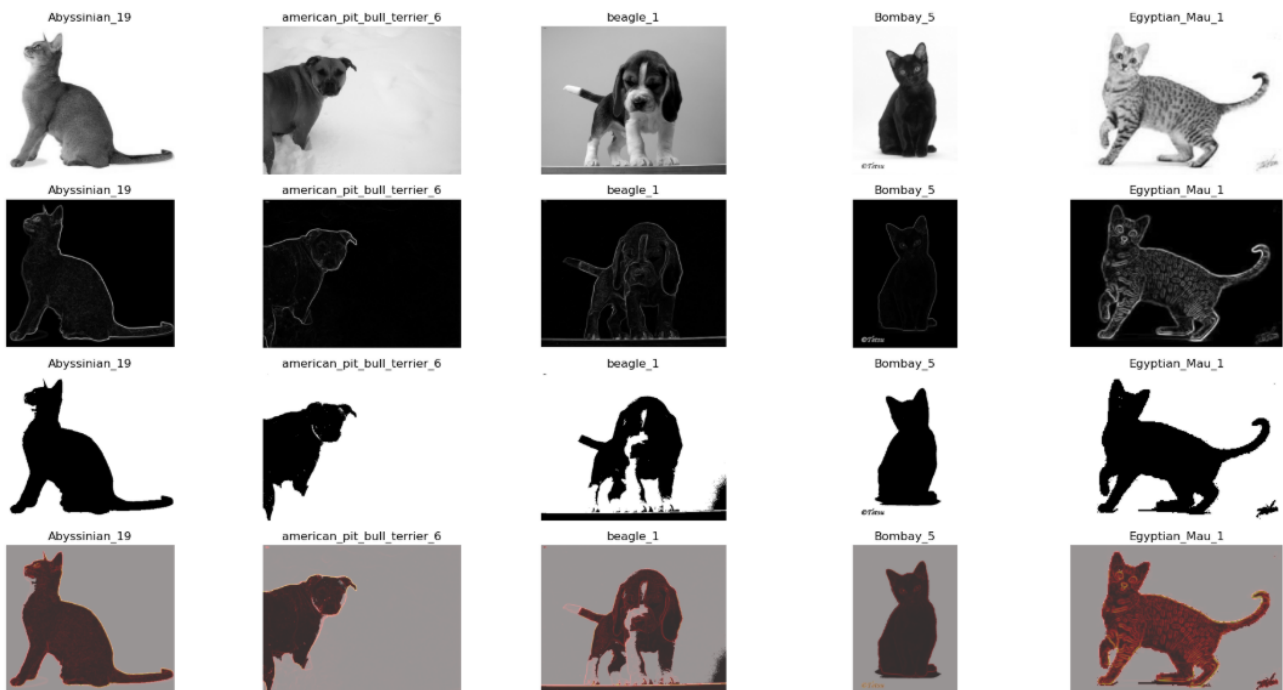
Each kernel is overlaid on a 3x3 section of the image. The pixel values in this region are multiplied by the corresponding kernel values then summed. This new value represents the intensity gradient at the center pixel. With a sliding window approach, the kernel is moved throughout the entire image. Areas with large gradient values produce higher kernel values which show that there is higher intensity differences between neighboring pixels. We can then apply a threshold to filter for only significant edges.

In addition to this, we can apply a threshold technique to segment the image by manually filtering out pixels that less than the threshold value. While this technique is not computationally expensive, it only works well when the pixel intensities between the object and background are very different.

To find reasonable threshold values, a histogram plotting the pixel intensities over frequencies was plotted, and hand picked values were chosen based on the peaks the graph created.



The red line depicts the chosen threshold value for provided image, the left side of the line are the pixel intensities inside the image that are within the segmented region while the pixel intensities that are outside are set as the background.



Dice and IoU metrics showed that these methods proved to be effective for segmentation on images that have a distinct solid background.

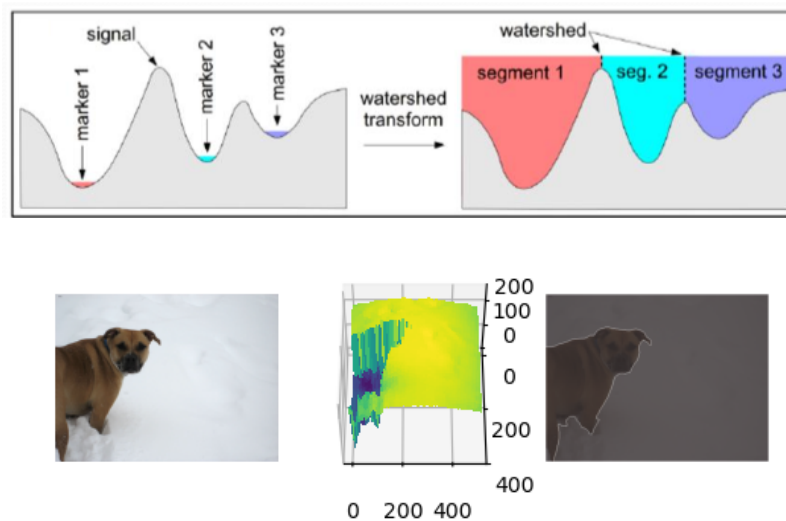
```
Abyssinian_19 Dice Coefficient: 0.8074
american_pit_bull_terrier_6 Dice Coefficient: 0.9012
beagle_1 Dice Coefficient: 0.8369
Bombay_5 Dice Coefficient: 0.7985
Egyptian_Mau_1 Dice Coefficient: 0.8099
-----
Abyssinian_19 IoU: 0.6771
american_pit_bull_terrier_6 IoU: 0.8201
beagle_1 IoU: 0.7195
Bombay_5 IoU: 0.6645
Egyptian_Mau_1 IoU: 0.6805
```

While these methods are effective at separating the object from the background, it only provides instance segmentation. Additionally, a majority of the full segmented map was due to threshold rather than the sobel edge detection.

Region-Based Segmentation

Watershed segmentation treats the image as a 3D surface, where higher gradients create peaks while lower gradients form valleys. The algorithm floods the surface starting from predefined markers. As the regions expand, boundaries are formed along the ridges where each segment touch.

This is similar to region growing techniques, but instead of a singular seed, watershed can handle multiple predefined markers to grow multiple regions simultaneously and determine their boundaries.



To define the markers, sure foreground is obtained by computing the distance from each foreground pixel to the nearest background pixel, then thresholding that distance map to only use the high confidence foreground pixels. To find the sure background, the thresholded binary image map is dilated. Finally a unknown region is defined by the area



between the sure background and sure foreground. Then marker labels are assigned to each segmented part, so each separated region receives a unique marker value.

```
Abyssinian_19 Watershed IoU: 0.9786
american_pit_bull_terrier_6 Watershed IoU: 0.9870
beagle_1 Watershed IoU: 0.9850
Bombay_5 Watershed IoU: 0.9895
Egyptian_Mau_1 Watershed IoU: 0.9581
-----
Abyssinian_19 Watershed Dice: 0.9892
american_pit_bull_terrier_6 Watershed Dice: 0.9934
beagle_1 Watershed Dice: 0.9924
Bombay_5 Watershed Dice: 0.9947
Egyptian_Mau_1 Watershed Dice: 0.9786
```

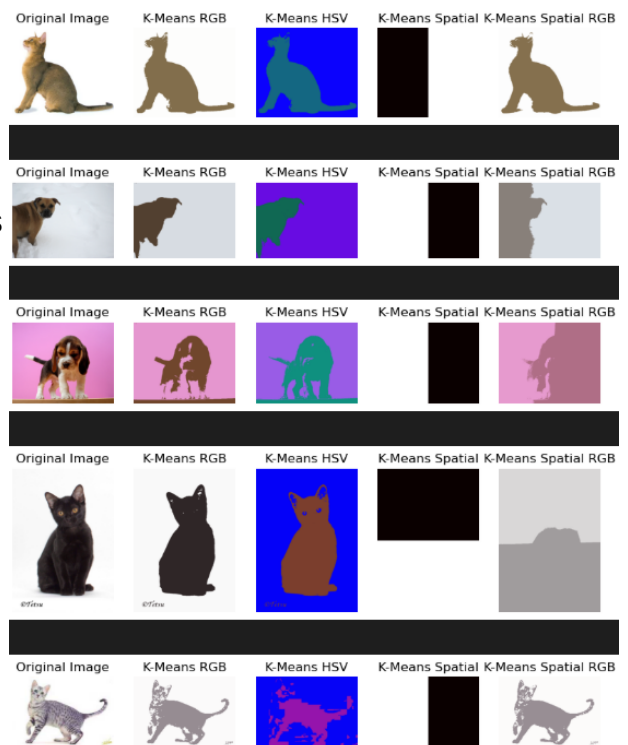
Watershed was effective at segmenting the image, achieving an average performance of 98% based on Dice metrics (as shown in evaluation results). The high contrast between the animal and the background allowed the algorithm to clearly identify object boundaries. The sure foreground was obtained by using a distance transform on the thresholded image, which helped define the most reliable foreground regions for marker generation.

Clustering-Based Segmentation

K-means segmentation was chosen for its simplicity and computational efficiency. During testing, mean shift I found it to be much more computationally expensive and provided similar results anyways.

K-means work by plotting all the pixels onto a n-dimensional plane. A set amount of points (centroids) are then placed randomly into the plane, each centroid represents a different class. Each value within the plane is then set to the class of the closest centroid, which essentially separates values in the n-dimensional plane into respective classes. Then each centroid is moved closer towards the average value of its respective class, and the process repeats.

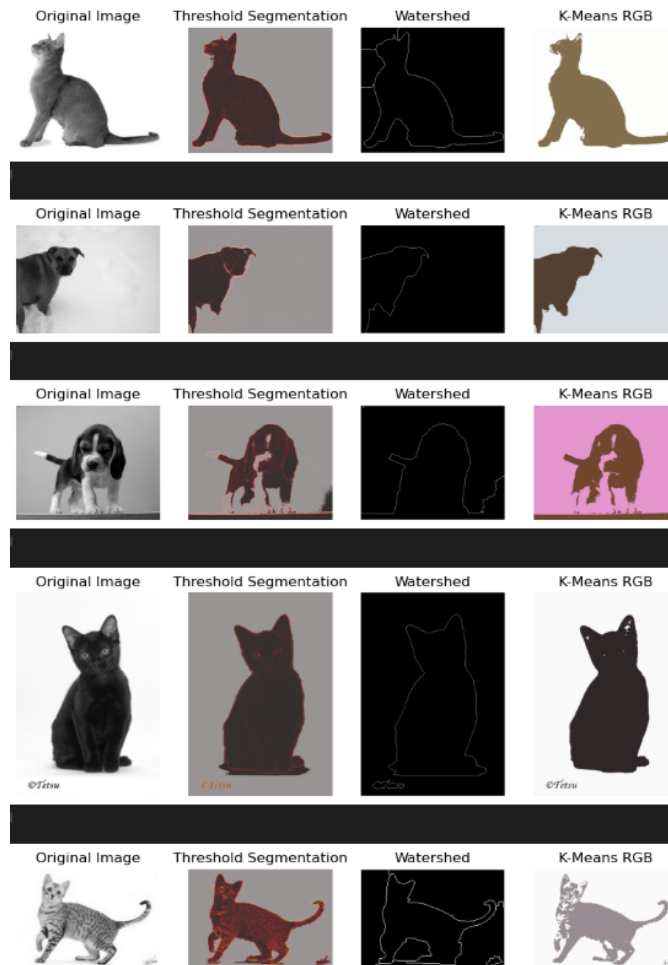
As k-means handles different features (RGB values, HSV values and spatial values), I experimented with the different types to find the most effective feature space. As spatial features only took the x & y location into account for segmentation, it doesn't provide any useful information for segmentation, therefore a combination of both spatial and RGB values was considered as it provided much better results.



```
Abyssinian_19 RGB IoU: 0.7891, Dice: 0.8821
Abyssinian_19 HSV IoU: 0.7932, Dice: 0.8847
Abyssinian_19 Spatial IoU: 0.3676, Dice: 0.5375
Abyssinian_19 Spatial RGB IoU: 0.7754, Dice: 0.8735
-----
american_pit_bull_terrier_6 RGB IoU: 0.6855, Dice: 0.8134
american_pit_bull_terrier_6 HSV IoU: 0.6848, Dice: 0.8130
american_pit_bull_terrier_6 Spatial IoU: 0.2597, Dice: 0.4123
american_pit_bull_terrier_6 Spatial RGB IoU: 0.3725, Dice: 0.5428
-----
beagle_1 RGB IoU: 0.5635, Dice: 0.7208
beagle_1 HSV IoU: 0.6361, Dice: 0.7776
beagle_1 Spatial IoU: 0.2273, Dice: 0.3704
beagle_1 Spatial RGB IoU: 0.3271, Dice: 0.4930
-----
Bombay_5 RGB IoU: 0.8470, Dice: 0.9172
Bombay_5 HSV IoU: 0.8336, Dice: 0.9092
Bombay_5 Spatial IoU: 0.2743, Dice: 0.4305
Bombay_5 Spatial RGB IoU: 0.3282, Dice: 0.4942
-----
Egyptian_Mau_1 RGB IoU: 0.6879, Dice: 0.8151
Egyptian_Mau_1 HSV IoU: 0.5008, Dice: 0.6674
Egyptian_Mau_1 Spatial IoU: 0.2461, Dice: 0.3950
Egyptian_Mau_1 Spatial RGB IoU: 0.7022, Dice: 0.8251
-----
```

RGB and HSV features usually provided higher IoU and Dice values compared to the spatial features. I believe this is due to the fact that spatial features don't really help with segmentation at all when the k value is so low and the only reason some spatial values scored so high is due to luck that the centroid happened to be placed in a good starting location.

Comparison



Watershed was the best performing segmentation method on the image set providing values in the high 90% while keeping computation to a minimum. I believe watershed worked so well due to the images having a high contrast from foreground to background allowing distance calculator to draw a good starting point. As watershed further thresholds the distances, it can create a more confident foreground which provides a better result. Pure thresholding with sobel edge detection was able to handle smaller gradients in the image which can be seen in the abyssinian where watershed produces some extra lines near the face of the cat. K-means performed well, but struggled with the detail in the egyptian mau. While K-means didn't require a predefined threshold value, it was much more computationally expensive compared to the other methods, and I don't really see much of a use case for this method compared to others. Especially with k-means on just spatial information.

Given all the segmentation methods, watershed performed the best as it provided great results while keeping computation down. Additionally I didn't need to do any manual calculation to find the best threshold value, as the predefined 20% max cut off value worked well.